

IT6130 · LẬP TRÌNH VÀ ẢO HÓA CHO IoT — BUỔI 2

Điều khiển thiết bị từ Cloud

& xây dựng ứng dụng IoT hoàn chỉnh

RPC

REST API

Dashboard

Lab 2

Đặng Tuấn Linh

School of Information and Communications Technology (SoICT)
Hanoi University of Science and Technology

3 tiếng · 35 slide · 1 buổi Lab

1

BUỔI 2

Lộ trình 3 tiếng hôm nay

1

Ôn buổi 1 & giao tiếp hai chiều

Nhìn lại uplink, giới thiệu downlink — cloud gửi lệnh xuống thiết bị.

2

RPC và điều khiển thiết bị

Cơ chế để cloud ra lệnh: one-way / two-way, luồng và topic MQTT.

3

REST API và tích hợp ứng dụng

JWT, lấy deviceId, gửi RPC từ ứng dụng qua HTTP.

4

Thực hành Lab 2 — Bài tập 4

Code Python hai phía: thiết bị (4b) và ứng dụng (4a), chạy end-to-end.

5

Dashboard & hoàn thiện hệ thống

Widget hiển thị & điều khiển, alarm, Rule Engine, tổng kết.

IT6130 · Lập trình và ảo hóa cho IoT — Buổi 2

2

2

NHÌN LẠI

Buổi 1 chúng ta đã làm được gì?



Đăng ký thiết bị

Provision qua MQTT với username "provision" → nhận ACCESS_TOKEN định danh thiết bị.



Telemetry qua MQTT

Token làm username, publish JSON lên topic v1/devices/me/telemetry theo chu kỳ.



Telemetry qua HTTP

Cùng mục tiêu nhưng dùng requests.post() — đơn giản hơn, tốn tài nguyên hơn.

Tất cả mới chỉ là chiều LÊN. Như căn nhà có cảm biến nhưng chưa có công tắc — hôm nay ta làm chiều XUỐNG: điều khiển.

3

BUỔI 2

Hôm nay chúng ta sẽ làm gì?



Giao tiếp 2 chiều

Cloud không chỉ nhận — còn gửi lệnh xuống điều khiển thiết bị.



Hiểu & dùng RPC

One-way & two-way giữa server và thiết bị.



Dùng REST API

Ứng dụng/Frontend gọi API lấy thiết bị, gửi lệnh.



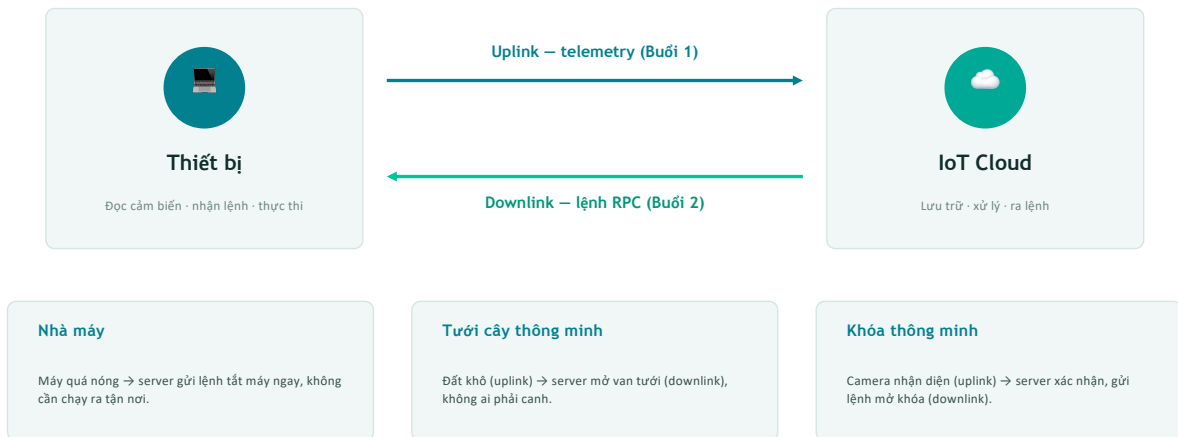
Dashboard & ứng dụng

Hiển thị, cảnh báo, điều khiển trực quan.

6

KHÁI NIỆM

Giao tiếp hai chiều: Uplink & Downlink



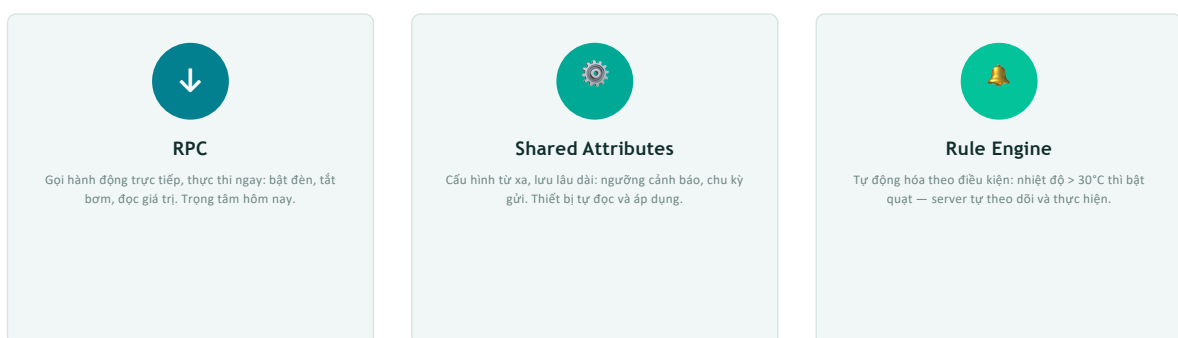
IT6130 · Lập trình và ảo hóa cho IoT — Buổi 2

5

7

KHÁI NIỆM

Server tương tác với thiết bị bằng cách nào?



RPC là "hành động ngay bây giờ" · Attributes là "cấu hình từ đây về sau" · Rule Engine là "robot tự làm".

IT6130 · Lập trình và ảo hóa cho IoT — Buổi 2

6

10

RPC

RPC — Remote Procedure Call là gì?

Gọi một hàm không nằm trên máy mình — mà trên một thiết bị khác, qua mạng — và nó thực thi ở đó.



Bật / tắt đèn

`setLight(true)`



Đổi tần suất gửi

`setInterval(1000)`



Đọc giá trị hiện tại tại

`getTemperature()`

Cấu trúc một lệnh RPC

method — tên hành động, do lập trình viên TỰ ĐẶT, hai phía phải khớp chính xác.

params — tham số truyền vào: true/false, số, chuỗi hay object JSON.

IT6130 · Lập trình và ảo hóa cho IoT — Buổi 2

7

11

RPC

One-way vs Two-way RPC



One-way — gửi và quên

- Server gửi lệnh, KHÔNG chờ thiết bị trả lời.
- Nhanh, đơn giản, ít tốn tài nguyên.
- Dùng khi chỉ cần ra lệnh: bật đèn, mở khóa.
- Giống remote điều hòa — bấm là xong.



Two-way — gửi và chờ kết quả

- Server gửi lệnh và CHỜ thiết bị trả kết quả.
- Biết được lệnh có thành công hay không.
- Dùng khi cần dữ liệu về: đọc nhiệt độ, hỏi trạng thái.
- Giống gọi điện hỏi — phải đợi câu trả lời.

Chọn theo nhu cầu: chỉ cần ra lệnh → one-way · cần kết quả về → two-way.

IT6130 · Lập trình và ảo hóa cho IoT — Buổi 2

8

13

RPC

Luồng RPC trong ThingsBoard



Với two-way: kết quả đi ngược lại theo đúng đường, về tới ứng dụng đã gửi lệnh.

15

RPC

Topic MQTT cho RPC (phía thiết bị)

Thiết bị kết nối MQTT với username = ACCESS_TOKEN, rồi lắng nghe lệnh trên topic request.

Hai topic quan trọng

Subscribe `v1/devices/me/rpc/request/+`

Trả lời `v1/devices/me/rpc/response/$request_id`

- Dấu + là wildcard — subscribe một lần, nhận mọi lệnh bất kể request_id.
- Mỗi lệnh tới topic cụ thể: `.../rpc/request/5` — số 5 chính là request_id.
- Trả lời two-way phải dùng ĐÚNG request_id đó để server ghép cặp lệnh ↔ kết quả (như ghi số bàn lên món ăn).

16

MỞ RỘNG

Shared Attributes — cấu hình từ xa

- Tham số cấu hình do server đặt cho thiết bị.
- Thiết bị đọc được và nhận thông báo khi giá trị thay đổi.
- Lưu lại trong ThingsBoard — thiết bị mất điện, bật lại vẫn đọc được.
- VD: ngưỡng cảnh báo, chu kỳ gửi dữ liệu.
- Đổi cấu hình 50 thiết bị: cập nhật 1 attribute thay vì gửi 50 lệnh RPC.

RPC hay Attributes?

RPC: hành động ngay (bật đèn)

Attributes: cấu hình lâu dài

RPC: tức thời, không lưu lại

Attributes: lưu lại, thiết bị đọc khi cần

Câu hỏi kiểm tra: đổi chu kỳ gửi từ 2s lên 30s để tiết kiệm pin — dùng gì? → Attributes (cấu hình bên).

18

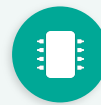
RPC

Server-side vs Client-side RPC



Server-side RPC

Server CHỦ ĐỘNG gửi lệnh xuống thiết bị. Dùng để điều khiển.



Client-side RPC

Thiết bị CHỦ ĐỘNG hỏi server (vd: xin giờ, xin cấu hình).

Lab hôm nay dùng server-side RPC — server điều khiển thiết bị.

20

RPC

Tóm tắt: chọn cơ chế phù hợp

Nhu cầu	Cơ chế nên dùng
Ra lệnh tức thì, không cần kết quả	One-way RPC
Ra lệnh + cần biết kết quả	Two-way RPC
Đặt cấu hình lâu dài cho thiết bị	Shared Attributes
Tự động hóa theo điều kiện	Rule Engine

21

REST API

REST API của ThingsBoard

- Cách để ứng dụng (web, script, app) tương tác với nền tảng.
- Tài liệu tương tác qua Swagger UI — xem và thử API ngay trên trình duyệt.
- Hầu hết API yêu cầu xác thực bằng JWT token (token của TÀI KHOẢN, khác ACCESS_TOKEN của thiết bị).
- Dùng để: quản lý thiết bị, đọc telemetry, gửi RPC, tạo dashboard...

4 endpoint dùng hôm nay

```

POST /api/auth/login
  - đăng nhập, lấy JWT

GET /api/tenant/devices
  - danh sách thiết bị

POST /api/rpc/oneway/{id}
  - gửi lệnh, không chờ

POST /api/rpc/twoway/{id}
  - gửi lệnh, chờ kết quả

```

Tương tự: gọi API của Google Maps hay bất kỳ dịch vụ web nào — request HTTP + JSON + header xác thực.

22

REST API

Xác thực bằng JWT token

- 1 POST username + password tới /api/auth/login.
- 2 Server trả về JSON chứa trường "token" — đó là JWT.
- 3 Lưu JWT lại để dùng cho các request sau.
- 4 Đính vào header mỗi request: X-Authorization: Bearer <jwt>.

JWT như thẻ phòng khách sạn: check-in được phát thẻ, dùng thẻ mở cửa, hết hạn thì làm thẻ mới (đăng nhập lại).

Quên headers=h ở bất kỳ request nào → lỗi 401 Unauthorized ngay. JWT có thời hạn — hết hạn thì đăng nhập lại.

Đăng nhập lấy JWT — Python

```
import requests
BASE = "https://thingsboard.cloud"

r = requests.post(
    f"{BASE}/api/auth/login",
    json={"username": "u@x.com",
          "password": "pass"})
jwt = r.json()["token"]

# header cho mọi request sau
h = {"X-Authorization":
     f"Bearer {jwt}"}
```

REST API

Lấy danh sách thiết bị & device ID

Để gửi RPC tới một thiết bị, cần biết deviceId của nó. Lấy qua API danh sách thiết bị.

```
h = {"X-Authorization": f"Bearer {jwt}"

r = requests.get(
    f"{BASE}/api/tenant/devices?page=0"
    "&pageSize=10", headers=h)

for d in r.json()["data"]:
    print(d["name"], d["id"]["id"])
```

Lưu ý

- Kết quả phân trang (page/pageSize)
- deviceId nằm ở d["id"]["id"]
- Lưu lại id để gửi RPC

REST API

Gửi RPC qua REST API

```
dev = "DEVICE_ID"
body = {"method": "setLight",
        "params": True}

# one-way: không chờ kết quả
requests.post(
    f"{BASE}/api/rpc/one-way/{dev}",
    json=body, headers=h)

# two-way: đổi 'one-way' -> 'two-way'
# và đọc response trả về
```

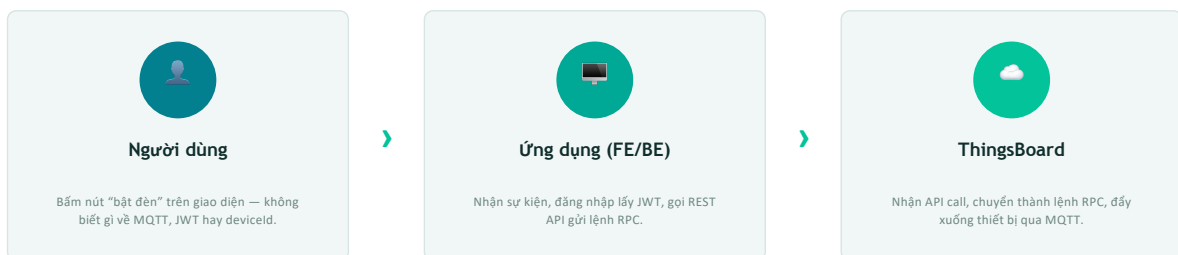
Cấu trúc lệnh

- method: tên hành động
- params: tham số đi kèm
- oneway: gửi & quên
- twoway: chờ kết quả

27

TÍCH HỢP

Ứng dụng gọi API như thế nào?



Ứng dụng là lớp trung gian: người dùng không bao giờ chạm tới token hay ThingsBoard. Hôm nay script Python đóng vai ứng dụng; thực tế là nút bấm trên web.

28

TÍCH HỢP

Bảo mật API & token — lưu ý thực tế



Không lộ token

Không đưa ACCESS_TOKEN/JWT vào code công khai hay client.



Token hết hạn

JWT có thời hạn → cần làm mới (refresh) khi cần.



Dùng HTTPS

Luôn gọi API qua HTTPS để mã hóa đường truyền.



Phân quyền

Mỗi tài khoản/thiết bị chỉ truy cập đúng phạm vi.

30

LAB 2 · THỰC HÀNH

Bài tập 4 — Điều khiển thiết bị qua RPC

Hai chương trình Python chạy song song và nói chuyện với nhau qua ThingsBoard.



device.py — thiết bị nhận lệnh (MQTT)



app.py — ứng dụng gửi lệnh (REST API)

Chuẩn bị: editor + 2 terminal + ACCESS_TOKEN từ buổi trước.

32

LAB 2 · TỔNG QUAN

Bài tập 4: hai phía của một hệ thống

4b

device.py — phía thiết bị

- Giả lập thiết bị IoT bằng Python.
- Kết nối MQTT, subscribe topic RPC.
- Nhận lệnh → in ra → trả kết quả (two-way).
- Chạy TRƯỚC, đứng chờ lệnh.

4a

app.py — phía ứng dụng

- Đóng vai ứng dụng điều khiển.
- Đăng nhập lấy JWT, lấy deviceId.
- Gửi lệnh RPC qua REST API.
- Chạy SAU, ở terminal thứ hai.

Thứ tự quan trọng: 4b chạy trước (thiết bị phải đứng sẵn chờ), rồi 4a gửi lệnh → quay lại terminal 4b xem lệnh được in ra.

LAB 2 · BÀI TẬP 4B

Thiết bị nhận lệnh RPC — nguyên lý

1

Kết nối MQTT với username = ACCESS_TOKEN (như Bài tập 2).

2

Subscribe topic v1/devices/me/rpc/request/+ — TRƯỚC khi server gửi lệnh.

3

Nhận lệnh trong callback on_message: đọc method & params, thực thi (lab: in ra màn hình; thiết bị thật: bật GPIO, relay...).

4

Nếu two-way: publish kết quả về .../rpc/response/\$id với đúng request_id.

Tin nhắn thiết bị nhận được

```
{
  "method": "setLight",
  "params": true
}
```

Topic: .../rpc/request/5 → request_id = 5

method cho biết hành động cần làm; params là dữ liệu kèm theo.

Thứ tự đúng trong code: connect → subscribe → loop. Subscribe muộn là có thể bỏ lỡ lệnh.

LAB 2 · BÀI TẬP 4B

Thiết bị nhận RPC — code Python

device.py

```
import paho.mqtt.client as mqtt, json
TOKEN = "sLzc0gDAZPkGMzFVTyUY"
REQ = "v1/devices/me/rpc/request/+"

def on_msg(c, u, m):
    rid = m.topic.split('/')[1]
    cmd = json.loads(m.payload)
    print("lệnh:", cmd["method"], cmd["params"])
    # ... thực thi hành động tại đây ...
    resp = f"v1/devices/me/rpc/response/{rid}"
    c.publish(resp, json.dumps({"ok": True}))

c = mqtt.Client()
c.username_pw_set(TOKEN)
c.on_message = on_msg
c.connect("thingsboard.cloud", 1883)
c.subscribe(REQ)
c.loop_forever()
```

Điểm chính

- rid: request_id tách từ topic — phần tử cuối sau split('/').
- on_message: callback gọi mỗi khi có lệnh tới.
- publish response: chỉ cần cho two-way.
- loop_forever(): giữ chương trình chạy chờ lệnh — terminal im lặng là BÌNH THƯỜNG.
- Thay TOKEN bằng token thật của bạn.

LAB 2 · BÀI TẬP 4A

Ứng dụng gửi lệnh RPC — nguyên lý

1. Đăng nhập /api/auth/login để lấy JWT token.
2. Lấy deviceId từ /api/tenant/devices.
3. Gửi POST /api/rpc/oneway/{deviceId} kèm method & params.
4. Hoặc /api/rpc/twoway/{deviceId} nếu cần chờ kết quả.

Phía ứng dụng (frontend/backend) là nơi phát lệnh — thường gắn với nút bấm trên giao diện.

LAB 2 · BÀI TẬP 4A

Gửi RPC qua REST — code Python

app.py

```
import requests
BASE = "https://thingsboard.cloud"

# 1) đăng nhập lấy token
jwt = requests.post(f"{BASE}/api/auth/login",
    json={"username": "u@x.com",
          "password": "pass"}).json()["token"]
h = {"X-Authorization": f"Bearer {jwt}"}

# 2) gửi lệnh RPC one-way
dev = "DEVICE_ID"
body = {"method": "setLight", "params": True}
requests.post(f"{BASE}/api/rpc/oneway/{dev}",
    json=body, headers=h)
print("đã gửi lệnh")
```

Ghép với 4b

- Chạy 4b trước — thiết bị đứng chờ.
- Chạy 4a để gửi lệnh.
- Quan sát terminal 4b in ra lệnh → điều khiển thành công.
- Thay username/password và DEVICE_ID bằng giá trị thật.

LAB 2 · KIỂM TRA

Chạy thử end-to-end



1 · Mở thiết bị

Chạy script 4b — thiết bị lắng nghe lệnh



2 · Gửi lệnh

Chạy script 4a hoặc bấm nút trên dashboard



3 · Quan sát

Thiết bị in lệnh & thực thi; two-way trả kết quả

LAB 2 · MỞ RỘNG

Nút điều khiển trên Dashboard

- ThingsBoard có widget dạng nút/công tắc.
- Cấu hình widget gửi RPC khi bấm (không cần code).
- Người dùng điều khiển thiết bị ngay trên trình duyệt.
- Kết hợp widget hiển thị để thấy phản hồi tức thì.

Ví dụ widget điều khiển

- Switch: bật/tắt thiết bị
- Button: gửi lệnh một chạm
- Knob: chỉnh giá trị (vd nhiệt độ)
- Round switch: trạng thái on/off

IT6130 · Lập trình và ảo hóa cho IoT

64

43

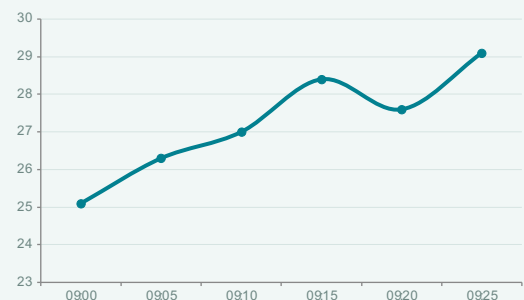
DASHBOARD

Hiển thị dữ liệu trên Dashboard

Dashboard biến số liệu thô thành thông tin trực quan, kéo-thả không cần lập trình.

- Data source: chọn thiết bị & khóa dữ liệu.
- Widget: thành phần hiển thị/điều khiển.
- Alias: nhóm thiết bị linh hoạt.

Nhiệt độ theo thời gian



IT6130 · Lập trình và ảo hóa cho IoT

65

44

DASHBOARD

Các loại widget thường dùng



Charts

Đường, cột, vùng — dữ liệu theo thời gian.



Gauges

Đồng hồ đo giá trị hiện tại.



Control

Nút/công tắc gửi RPC điều khiển.



Maps

Vị trí thiết bị trên bản đồ.

IT6130 - Lập trình và ảo hóa cho IoT

66

45

HOÀN THIỆN

Cảnh báo (Alarm) theo ngưỡng

- Đặt điều kiện trên dữ liệu (vd nhiệt độ > 30°C).
- Khi vượt ngưỡng → tạo alarm tự động.
- Alarm có mức độ: cảnh báo, nghiêm trọng...
- Có thể gắn hành động: gửi email, kích RPC.

Ví dụ kịch bản

- Nhiệt độ > 30°C → bật quạt (RPC)
- Độ ẩm < 40% → cảnh báo
- Mất kết nối → báo inactive
- Pin yếu → thông báo bảo trì

IT6130 - Lập trình và ảo hóa cho IoT

67

46

HOÀN THIỆN

Rule Engine – tự động hóa luồng dữ liệu



Lọc & định tuyến

Phân loại dữ liệu đến theo điều kiện.



Biến đổi

Tính toán, làm giàu dữ liệu trước khi lưu.



Hành động

Tạo alarm, gửi RPC, gọi webhook...

Rule Engine xử lý dữ liệu tự động mà không cần viết server riêng.

IT6130 - Lập trình và ảo hóa cho IoT

68

47

HOÀN THIỆN

Một dashboard điển hình

Nhiệt độ (24h)



Độ ẩm

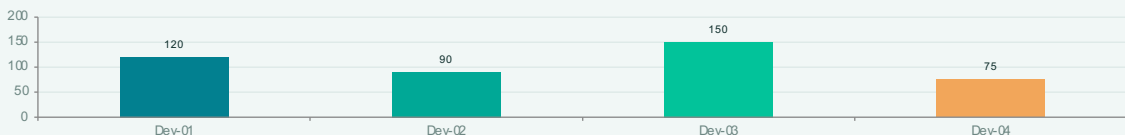
58%

Quạt



BẬT

Lượng dữ liệu theo thiết bị



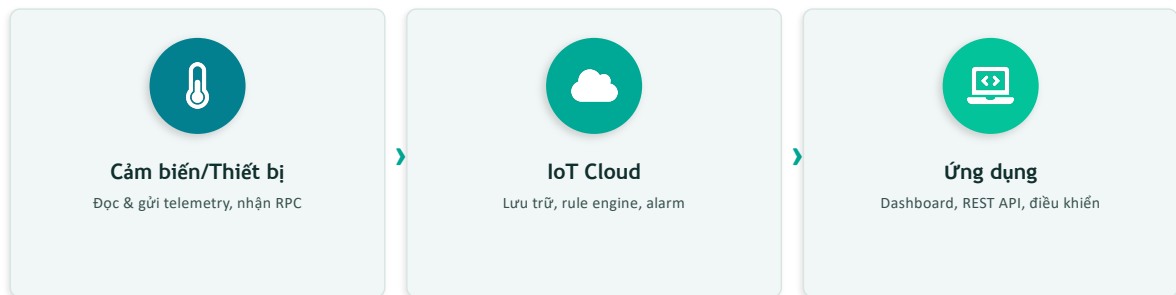
IT6130 - Lập trình và ảo hóa cho IoT

69

48

TỔNG KẾT

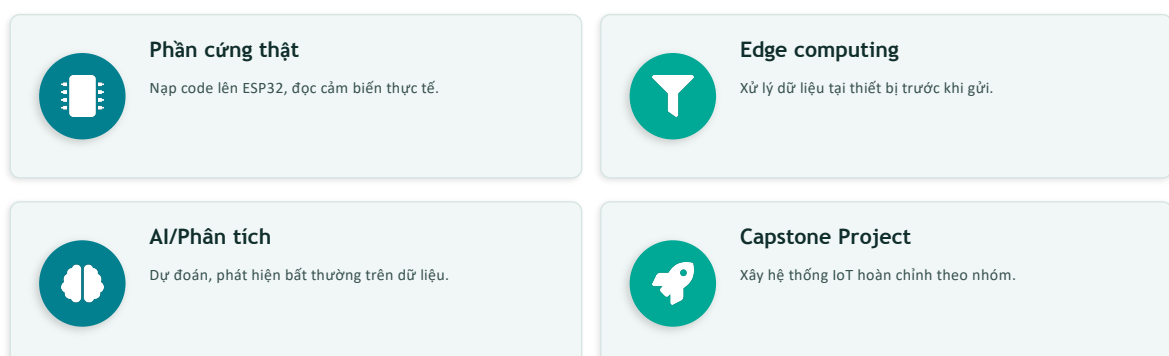
Bức tranh hệ thống IoT hoàn chỉnh



Hai chiều dữ liệu đã khép kín: thiết bị ↔ cloud ↔ ứng dụng.

TIẾP THEO

Hướng phát triển & Capstone Project



TỔNG KẾT 2 BUỔI**Hoàn thành vòng tròn giao tiếp IoT**

- ✓ Buổi 1: thiết bị đọc cảm biến & gửi dữ liệu lên cloud
- ✓ Buổi 2: cloud điều khiển thiết bị qua RPC & REST API
- ✓ Xây dashboard hiển thị, cảnh báo và điều khiển
- ✓ Sẵn sàng cho Capstone Project

Tài liệu tham khảo

- Buyya & Dastjerdi, "Internet of Things: Principles and Paradigms", Elsevier 2016.
- Tài liệu ThingsBoard: thingsboard.io/docs
- Antonio Liñán Colina et al., "IoT in 5 days", 2015.